

TURING

图灵程序设计丛书



Professional JavaScript for Web Developers, 4th Edition

JavaScript 高级程序设计 (第4版)

[美] 马特·弗里斯比 (Matt Frisbie) 著
李松峰 译



中国工信出版集团



人民邮电出版社
POSTS & TELECOM PRESS

作者简介

马特·弗里斯比 (Matt Frisbie)

知名前端技术专家，拥有十余年 Web 开发经验。曾是 Google 工程师，参与开发 Adsense 和 AMP 平台等重要产品。他也是美国外卖巨头 DoorDash 技术团队的工程师。目前担任 Gosellout 公司的 CTO。毕业于伊利诺伊大学厄巴纳-尚佩恩分校 (UIUC) 计算机专业。

译者简介

李松峰

360 前端开发资深专家、前端 TC 委员、W3C AC 代表，任职于“奇舞团”，也是 360 Web 字体服务“奇字库”作者。

数字版权声明

图灵社区的电子书没有采用专有客户端，您可以在任意设备上，用自己喜欢的浏览器和PDF阅读器进行阅读。

但您购买的电子书仅供您个人使用，未经授权，不得进行传播。

我们愿意相信读者具有这样的良知和觉悟，与我们共同保护知识产权。

如果购买者有侵权行为，我们可能对该用户实施包括但不限于关闭该帐号等维权措施，并可能追究法律责任。



图灵程序设计丛书

Professional JavaScript for Web Developers, 4th Edition

JavaScript 高级程序设计 (第4版)

[美] 马特·弗里斯比 (Matt Frisbie) 著
李松峰 译

人民邮电出版社
北 京

图书在版编目 (C I P) 数据

JavaScript高级程序设计 : 第4版 / (美) 马特·弗里斯比 (Matt Frisbie) 著 ; 李松峰译. -- 2版. -- 北京 : 人民邮电出版社, 2020.9 (2020.10重印)
(图灵程序设计丛书)
ISBN 978-7-115-54538-1

I. ①J… II. ①马… ②李… III. ①JAVA语言—程序设计 IV. ①TP312.8

中国版本图书馆CIP数据核字 (2020) 第134445号

ISBN: 9781119366447

Professional JavaScript for Web Developers, 4th Edition, by Matt Frisbie.

© 2020 by John Wiley & Sons, Inc., Indianapolis, Indiana.

本书简体中文版由 John Wiley & Sons, Inc. 授权人民邮电出版社独家出版。
本书封底贴有 John Wiley & Sons, Inc. 激光防伪标签, 无标签者不得销售。
版权所有, 侵权必究。

内 容 提 要

本书是 JavaScript 经典图书的新版。第4版涵盖 ECMAScript 2019, 全面、深入地介绍了 JavaScript 开发者必须掌握的前端开发技术, 涉及 JavaScript 的基础特性和高级特性。书中详尽讨论了 JavaScript 的各个方面, 从 JavaScript 的起源开始, 逐步讲解到新出现的技术, 其中重点介绍 ECMAScript 和 DOM 标准。在此基础上, 接下来的各章揭示了 JavaScript 的基本概念, 包括类、期约、迭代器、代理, 等等。另外, 书中深入探讨了客户端检测、事件、动画、表单、错误处理及 JSON。本书同时也介绍了近几年来涌现的重要新规范, 包括 Fetch API、模块、工作者线程、服务线程以及大量新 API。

本书适合有一定编程经验的 Web 应用程序开发人员阅读, 也可作为高校及社会实用技术培训相关专业课程的教材。

◆ 著 [美] 马特·弗里斯比

译 李松峰

责任编辑 温 雪

责任印制 周昇亮

◆ 人民邮电出版社出版发行 北京市丰台区成寿寺路11号

邮编 100164 电子邮件 315@ptpress.com.cn

网址 <https://www.ptpress.com.cn>

北京 印刷

◆ 开本: 800×1000 1/16

印张: 55.5

字数: 1710千字 2020年9月第2版

印数: 276 001—281 000册 2020年10月北京第2次印刷

著作权合同登记号 图字: 01-2019-7913号

定价: 129.00元

读者服务热线: (010)51095183转600 印装质量热线: (010)81055316

反盗版热线: (010)81055315

广告经营许可证: 京东市监广登字 20170147 号

献给 Jordan，感谢她无论听到多少次“快写完了”都仍然坚定地支持我。

译者序

七年弹指一挥间。2012 年到 2019 年是 JavaScript 蓬勃发展的七年，鼎鼎大名的 Stack Overflow 调查显示，截至 2019 年，JavaScript 已连续七年位居“最常用编程语言”（most commonly used programming language）榜首。事实上，2020 年的调查结果也毫无悬念，JavaScript 依旧独占鳌头。

2012 年是这本被誉为 JavaScript “红宝书”的著作第 3 版出版的时间。生逢其时，第 3 版狂销几十万册，影响深远，甚至改变了很多人的命运（包括本书译者）。随着 ECMAScript 2015（ES6）的发布，JavaScript 这门语言再次被注入新的生机与活力。2019 年 10 月，涵盖 ECMAScript 2019 的第 4 版面世。如今，跨过一个年头，中文版也要付梓了。

“红宝书”的这一版延续了上一版的框架和格局，删减了已经过时的内容，在此基础上又翔实地增补了 ES2015 到 ES2019 的全新内容，英文版篇幅也达到了前所未有的 1100 多页。

翻译期间，译者虽然尽最大努力确保译文准确、通顺，但错漏之处在所难免。为此特别感谢本书责任编辑温雪，感谢她对译稿认真细致的编辑和审校，以及对出版流程的卓越把控，确保了中文版的早日上市。

在本书印行前夕，为进一步确保出版质量、减少图书错误，我们邀请了数位一线前端开发工程师共同对本书进行了预读和勘误。在短短两周时间内，大家分工协作，筛查、发现并“消灭”了不少文字、排版、代码和技术上的问题，大大提升了本书首印质量。他们分别是（按审读章节顺序排序）饶占平、梁幸芝、陈方旭、林景宜、王欢、刘冰晶、邢洋洋、刘博文、刘观宇、王佳裕。特此致谢。特别感谢贺师俊（Hax）对“期约”（promise）及相关一系列术语翻译的建议。

最后，衷心祝愿罹患“莱姆病”（Lyme disease）的 Nicholas Zakas 早日康复。

2020 年 7 月 15 日

序

工业革命是钢铁铸就的，互联网革命则是 JavaScript 造就的。25 年的反复锻造与打磨，成就了 JavaScript 在今天的应用程序开发中毋庸置疑的统治地位，但并非一开始就是如此。

Brendan Eich 只用 10 天就写出了 JavaScript 的第一版。初生的 JavaScript 看似弱不禁风，但历史表明，第一印象并不代表一切。今天，这门语言的每个细节，也就是这本书所涉及的方方面面，都是反复推敲的产物。并非所有决定都让人满意，也没有完美的编程语言，不过单从无所不在这方面看，JavaScript 倒是很接近完美。它是目前唯一一个可以随处部署的语言：服务器、桌面浏览器、手机浏览器，甚至原生移动应用程序中都有它的身影。

JavaScript 目前的使用者有不同层次的软件工程师，他们的背景各异。无论是以开发设计精良、优雅的软件为目标，还是仅仅为了完成业绩而简单堆砌一个系统，JavaScript 都能派上用场。

怎么使用 JavaScript 完全取决于你。一切尽在你的掌握之中。

在我超过 15 年的软件开发生涯中，JavaScript 工具和最佳实践已经发生了天翻地覆的变化。2004 年，我开始接触这门语言，当时还是雅虎地球村（Geocities）、雅虎群组（Yahoo Groups）和 Macromedia Flash 播放器的天下。JavaScript 给人感觉像个玩具，当时我在 RSS、MySpace Profile Pages 等流行的沙盒环境中开始使用它。后来我又帮助一些个人网站修改和自定义功能，那种感觉就像在狂野的西部拓荒，而我也因此喜欢上了它。

当初我创建第一家公司的时候，配置主机装个数据库要花几天时间，而 JavaScript 只要扔到 HTML 里就可以跑起来。“前端应用程序”是不存在的，主要是零七碎八的函数。后来 Ajax 因为 jQuery 火了而变得更加流行，这才打开了通向新世界的大门，可靠、稳定的应用程序应运而生。这股风潮愈演愈烈，直到有一天遇到了发展瓶颈，但突然间，强大的框架诞生了。前端模型、数据绑定、路由管理、反应式视图，全都爆发出来了。我就在这个时候搬到硅谷，帮人打理一家公司。很快，使用我代码的用户达到了几百万。置身硅谷这么长时间以来，我也为开源做了一些贡献，培训了不计其数的软件工程师，也走了一点儿运。我的上一家公司在 2018 年被 Stripe 收购，我现在就供职于这家公司，致力于为互联网构建其经济基础设施。

我很高兴在马特第一次到帕洛阿尔托的一家小型创业公司领导工程化时结识了他。那家公司叫 Claco，当时我刚成为它的顾问。他追求伟大软件的活力和激情溢于言表，而这家羽翼未丰的公司很快就开发出一款漂亮的产品。一如为硅谷公司设立标杆的惠普，这家创业公司也诞生在一间平房里。但这

可不是寻常的民房，而是一间“黑客屋”，里面十几位才华横溢的软件工程师经常通宵达旦地工作。虽然过的不是什么高档次生活——他们坐的都是别人扔在大街上的那种沙发床和旧椅子——他们在这间房子里每天所写代码的数量和质量却引人瞩目。连续工作几小时后，大多数人会把精力投入到公司的另一个子项目上，然后又是几个小时的工作。不太会写代码的人也常受启发，发现自己学习的渴望，然后仅仅几个星期后就变成了代码能手。

马特是促成这种开发效率的关键角色。他是“黑客屋”里经验最丰富的人，恰好也是思维最清晰、最专业的一个。拿到计算机工程学位并不能说明什么，只要在窗户或者白板上看到马特写的算法、性能计算以及代码，你就知道马特又在专注于他的下一个大项目。随着我对他了解的加深，我们成为了好朋友。他的领悟能力，他对培训工作的热爱，以及几乎可以把所有东西转化成笑话的能力，都是我所欣赏的品质。

虽然马特是一位极具才华的软件工程师和项目领导，但他之所以能成为本书作者独一无二的人选，还是凭借他独有的经验和知识。

他不仅仅花时间教别人，而且还把这本书写完了。

在 Claco，他开发了多款整体性产品，端到端地帮助教师在课堂上提供更好的学习体验。在 DoorDash，他是第一位工程师，开发了一个可靠的物流配送系统并实现了高速增长，目前公司估值超过了 120 亿美元。最后，在 Google，马特写的软件已经被这个星球上的数十亿人使用了。

全情投入，快速增长，誉满天下——多数软件工程师终其一生也只能体验到其中一项，而且还得运气好。马特不仅体验到了全部，还成为了畅销书作者。除了本书，他还写了两本 JavaScript 和 Angular 的书。说实话，我就想知道他什么时候能写一本书，把自己管理时间的奥秘分享出来。

本书是一部翔实的工具书，满满的都是 JavaScript 知识和实用技术。我热切希望本书读者能够不断学习，并亲手打造属于自己的梦想。欢迎大家多多挑错，多记笔记，别忘了打开代码编辑器，毕竟互联网革命才刚刚开始！

Zach Tratar
Stripe 软件工程师
Jobstart 联合创始人、前 CEO

前言

关于 JavaScript，谷歌公司的一位技术经理曾经跟我分享过一个无法反驳的观点。他说 JavaScript 并不是一门真正有内聚力的编程语言，至少形式上不是。ECMA-262 规范定义了 JavaScript，但 JavaScript 没有唯一的实现。更重要的是，这门语言与其宿主关系密切。实际上宿主为 JavaScript 定义了与外界交互所需的全部 API：DOM、网络请求、系统硬件、存储、事件、文件、加密，还有数以百计的其他 API。各种浏览器及其 JavaScript 引擎都按照自己的理解实现了这些规范。Chrome 有 Blink/V8，Firefox 有 Gecko/SpiderMonkey，Safari 有 WebKit/JavaScriptCore，微软有 Trident/EdgeHTML/Chakra。浏览器以合规的方式运行绝大多数 JavaScript，但 Web 上随处可见迎合各种浏览器偏好的页面。因此，对 JavaScript 更准确的定位应该是一组浏览器实现。

Web 纯化论者可能认为 JavaScript 本身并非网页不可或缺的部分，但他们必须承认，如果没有 JavaScript，那么现代 Web 势必发生严重倒退。毫不夸张地讲，JavaScript 才是真正不可或缺的。如今，手机、计算机、平板设备、电视、游戏机、智能手表、冰箱，甚至连汽车都内置了可以执行 JavaScript 代码的 Web 浏览器。地球上有近 30 亿人在使用安装了 Web 浏览器的智能手机。这门语言迅速发展的社区催生了大量高质量的开源项目。浏览器也已经支持模拟原生移动应用程序的 API。Stack Overflow 2019 年的开发者调查显示，JavaScript 连续七年位于最流行编程语言榜首。

我们正迎来 JavaScript 的文艺复兴。

本书将从 JavaScript 的起源讲起，从最初的 Netscape 浏览器直到今天各家浏览器支持的让人眼花缭乱的技术。全书对大量高级技术进行了鞭辟入里的剖析，以确保读者真正理解这些技术并掌握它们的应用场景。简而言之，通过学习本书，读者可以透彻地理解如何选择恰当的 JavaScript 技术，以解决现实开发中遇到的业务问题。

读者对象

本书适合以下读者阅读。

- ❑ 有经验的开发者，熟悉面向对象编程，因为 JavaScript 与 Java 和 C++ 等传统面向对象（OO，Object Oriented）语言的关系而希望学习 JavaScript。
- ❑ Web 应用程序开发者，希望增强自己的网站或 Web 应用程序的易用性。

❑ 初级 JavaScript 开发者，希望更好地理解这门语言。

此外，熟悉以下相关技术对阅读本书非常有帮助。

- ❑ Java
- ❑ PHP
- ❑ Python
- ❑ Ruby
- ❑ Golang
- ❑ HTML
- ❑ CSS

本书内容

本书第 4 版全面深入地介绍了 JavaScript 开发者必须掌握的前端开发技术，涉及 JavaScript 的基础特性和高级特性。

本书从 JavaScript 的起源开始，逐步讲解到今天的最新技术。书中详尽讨论了 JavaScript 的各个方面，重点介绍 ECMAScript 和 DOM 标准。

在此基础上，接下来的各章揭示了 JavaScript 的基本概念，包括类、期约、迭代器、代理，等等。另外，书中还深入探讨了客户端检测、事件、动画、表单、错误处理及 JSON。

本书最后介绍近几年来涌现的最新和最重要的规范，包括 Fetch API、模块、工作者线程、服务线程以及大量新 API。

组织结构

本书包含如下这些章。多个章节配有免费视频课二维码，扫描即可观看。

第 1 章，介绍 JavaScript 的起源：从哪里来，如何发展，以及现今的状况。这一章会谈到 JavaScript 与 ECMAScript 的关系、DOM、BOM，以及 Ecma 和 W3C 相关的标准。

第 2 章，了解 JavaScript 如何与 HTML 结合起来创建动态网页，主要介绍在网页中嵌入 JavaScript 的不同方式，还有 JavaScript 的内容类型及其与<script>元素的关系。

第 3 章，介绍语言的基本概念，包括语法和流控制语句；解释 JavaScript 与其他类 C 语言在语法上的异同点。在讨论内置操作符时也会谈到强制类型转换。此外还将介绍所有的原始类型，包括 Symbol。

第 4 章，探索 JavaScript 松散类型下的变量处理。这一章将涉及原始类型与引用类型的不同，以及与变量有关的执行上下文。此外，这一章也会讨论 JavaScript 中的垃圾回收，涉及在变量超出作用域时如何回收内存。

第 5 章，讨论 JavaScript 所有内置的引用类型，如 Date、Regexp、原始类型及其包装类型。每种

引用类型既有理论上的讲解，也有相关浏览器实现的剖析。

第 6 章，继续讨论内置引用类型，包括 `Object`、`Array`、`Map`、`WeakMap`、`Set` 和 `WeakSet` 等。

第 7 章，介绍 ECMAScript 新版中引入的两个基本概念：迭代器和生成器，并分别讨论它们最基本的行为和在当前语言环境下的应用。

第 8 章，解释如何在 JavaScript 中使用类和面向对象编程。首先会深入讨论 JavaScript 的 `Object` 类型，进而探讨原型式继承，接下来全面介绍 ES6 类及其与原型式继承的紧密关系。

第 9 章，介绍两个紧密相关的概念：`Proxy`（代理）和 `Reflect`（反射）API。代理和反射用于拦截和修改这门语言的基本操作。

第 10 章，探索 JavaScript 最强大的一个特性：函数表达式，主要涉及闭包、`this` 对象、模块模式、创建私有对象成员、箭头函数、默认参数和扩展操作符。

第 11 章，介绍两个紧密相关的异步编程构造：`Promise` 类型和 `async/await`。这一章讨论 JavaScript 的异步编程范式，进而介绍期约（`promise`）与异步函数的关系。

第 12 章，介绍 BOM，即浏览器对象模型，跟与浏览器本身交互的 API 相关。所有 BOM 对象都会涉及，包括 `window`、`document`、`location`、`navigator` 和 `screen` 等。

第 13 章，解释检测客户端机器及其能力的不同手段，包括能力检测和用户代理字符串检测。这一章讨论每种手段的优缺点，以及适用的场景。

第 14 章，介绍 DOM，即文档对象模型，主要是 DOM Level 1 定义的 API。这一章将简单讨论 XML 及其与 DOM 的关系，进而全面探索 DOM 以及如何利用它操作网页。

第 15 章，解释其他 DOM API，包括浏览器本身对 DOM 的扩展，主要涉及 `Selectors API`、`Element Traversal API` 和 HTML5 扩展。

第 16 章，在之前两章的基础上，解释 DOM Level 2 和 Level 3 对 DOM 的扩展，包括新增的属性、方法和对象。这一章还会介绍 DOM4 的相关内容，比如 `Mutation Observer`。

第 17 章，解释事件在 JavaScript 中的本质，以及事件的起源及其在 DOM 中的运行方式。

第 18 章，围绕 `<canvas>` 标签讨论如何创建动态图形，包括 2D 和 3D 上下文（`WebGL`）等动画和游戏开发所需的基础。这一章还会讨论 `WebGL1` 和 `WebGL2`。

第 19 章，探索使用 JavaScript 增强表单交互及突破浏览器限制，主要讨论文本框、选择框等表单元素及数据验证和操作。

第 20 章，介绍各种 JavaScript API，包括 `Atomics`、`Encoding`、`File`、`Blob`、`Notifications`、`Streams`、`Timing`、`Web Components` 和 `Web Cryptography`。

第 21 章，讨论浏览器如何处理 JavaScript 代码中的错误及几种错误处理方式。这一章同时介绍了每种浏览器的调试工具和技术，包括简化调试过程的建议。

第 22 章, 介绍通过 JavaScript 读取和操作 XML 数据的特性, 解释了不同浏览器支持特性和对象的差异, 提供了简化跨浏览器编码的建议。这一章也讨论了使用 XSLT 在客户端转换 XML 数据。

第 23 章, 介绍作为 XML 替代的 JSON 数据格式, 还讨论了浏览器原生解析和序列化 JSON, 以及使用 JSON 时要注意的安全问题。

第 24 章, 探讨浏览器请求数据和资源的常用方式, 包括早期的 XMLHttpRequest 和现代的 Fetch API。

第 25 章, 讨论应用程序离线时在客户端机器上存储数据的各种技术。先从 cookie 谈起, 然后讨论 Web Storage 和 IndexedDB。

第 26 章, 介绍模块模式在编码中的应用, 进而讨论 ES6 模块之前的模块加载方式, 包括 CommonJS、AMD 和 UMD。最后介绍新的 ES6 模块及其正确用法。

第 27 章, 深入介绍专用工作者线程、共享工作者线程和服务工作者线程。其中包括工作者线程在操作系统和浏览器层面的实现, 以及使用各种工作者线程的最佳策略。

第 28 章, 探讨在企业级开发中进行 JavaScript 编码的最佳实践。其中提到了提升代码可维护性的编码惯例, 包括编码技巧、格式化及通用编码建议。深入讨论应用性能和提升速度的技术。最后介绍与上线部署相关的话题, 包括项目构建流程。

预备条件

要运行本书示例代码, 需要如下条件。

- ❑ 现代操作系统, 包括 Windows、Linux、Mac OS、Android 或 iOS。
- ❑ 现代浏览器, 如 IE11+、Edge 12+、Firefox 26+、Chrome 39+、Safari 10+、Opera 26+ 或 iOS Safari 10+。

扫描封底二维码, 可以下载本书源代码, 并加入图灵前端研发小组。^①

电子书及附录

扫描下方二维码, 即可购买本书中文版电子书, 并从“随书下载”处获取本书电子版附录。



^① 读者也可访问本书图灵社区页面 (<https://www.ituring.com.cn/book/2472>) 下载本书配套学习资源, 并提交中文版勘误。

——编者注

致 谢

感谢 Wiley 出版社让我接手这本书。编写本书第 4 版对我来说是前所未有的挑战，也让我收获非常大。来自 Wiley 的包容和支持是本书得以完成的前提。感谢 Wiley 的工作人员，特别是把这本书交到我手上并紧盯着整个流程的 Jim Minatel。

感谢本书前 3 版的作者 Nicholas C. Zakas，感谢他在我接手之前所做的一切。没有他之前打下的良好基础，就不会有本书今天的成就。衷心祝愿他早日康复。

特别感谢 Adaobi Obi Tulton 的指导。如果没有她对整个流程的把控，以及她的耐心和专业水准，我不可能写完这一版。

还要感谢对本书草稿给出反馈意见的所有人：Samuel Kallner、Chaim Krause、Marcia Wilbur、Nancy Rapoport、Athyappan Lalith Kumar，还有 Evelyn Wellborn。这样一本书，少了你们任何人的帮助，都不会像现在这么完善。

最后，我想感谢 Zach Tratar 为本书作序。我非常幸运地在搬到旧金山的头一天就认识了 Zach Tratar。几年来，作为良师益友，他的求知若渴和博学多才一直感染着我，何况他还是一位杰出的软件工程师。他同意为本书作序是我的荣幸。

目 录

第 1 章 什么是 JavaScript	1
1.1 简短的历史回顾	1
1.2 JavaScript 实现	2
1.2.1 ECMAScript	2
1.2.2 DOM	6
1.2.3 BOM	8
1.3 JavaScript 版本	9
1.4 小结	10
第 2 章 HTML 中的 JavaScript	11
2.1 <script>元素	11
2.1.1 标签位置	13
2.1.2 推迟执行脚本	14
2.1.3 异步执行脚本	14
2.1.4 动态加载脚本	15
2.1.5 XHTML 中的变化	16
2.1.6 废弃的语法	17
2.2 行内代码与外部文件	18
2.3 文档模式	18
2.4 <noscript>元素	19
2.5 小结	20
第 3 章 语言基础	21
3.1 语法	21
3.1.1 区分大小写	21
3.1.2 标识符	21
3.1.3 注释	22
3.1.4 严格模式	22
3.1.5 语句	22
3.2 关键字与保留字	23

3.3 变量	24
3.3.1 var 关键字	24
3.3.2 let 声明	25
3.3.3 const 声明	28
3.3.4 声明风格及最佳实践	29
3.4 数据类型	30
3.4.1 typeof 操作符	30
3.4.2 Undefined 类型	30
3.4.3 Null 类型	32
3.4.4 Boolean 类型	33
3.4.5 Number 类型	33
3.4.6 String 类型	38
3.4.7 Symbol 类型	44
3.4.8 Object 类型	56
3.5 操作符	56
3.5.1 一元操作符	56
3.5.2 位操作符	59
3.5.3 布尔操作符	64
3.5.4 乘性操作符	66
3.5.5 指数操作符	67
3.5.6 加性操作符	68
3.5.7 关系操作符	69
3.5.8 相等操作符	70
3.5.9 条件操作符	72
3.5.10 赋值操作符	72
3.5.11 逗号操作符	73
3.6 语句	73
3.6.1 if 语句	73
3.6.2 do-while 语句	74
3.6.3 while 语句	74

3.6.4 for 语句	74	5.3.2 Number	115
3.6.5 for-in 语句	75	5.3.3 String	117
3.6.6 for-of 语句	76	5.4 单例内置对象	128
3.6.7 标签语句	76	5.4.1 Global	128
3.6.8 break 和 continue 语句	76	5.4.2 Math	132
3.6.9 with 语句	78	5.5 小结	135
3.6.10 switch 语句	78	第 6 章 集合引用类型	136
3.7 函数	80	6.1 Object	136
3.8 小结	82	6.2 Array	138
第 4 章 变量、作用域与内存	83	6.2.1 创建数组	138
4.1 原始值与引用值	83	6.2.2 数组空位	140
4.1.1 动态属性	83	6.2.3 数组索引	141
4.1.2 复制值	84	6.2.4 检测数组	142
4.1.3 传递参数	85	6.2.5 迭代器方法	143
4.1.4 确定类型	86	6.2.6 复制和填充方法	143
4.2 执行上下文与作用域	87	6.2.7 转换方法	145
4.2.1 作用域链增强	89	6.2.8 栈方法	147
4.2.2 变量声明	90	6.2.9 队列方法	147
4.3 垃圾回收	94	6.2.10 排序方法	148
4.3.1 标记清理	95	6.2.11 操作方法	150
4.3.2 引用计数	95	6.2.12 搜索和位置方法	151
4.3.3 性能	96	6.2.13 迭代方法	153
4.3.4 内存管理	97	6.2.14 归并方法	154
4.4 小结	101	6.3 定型数组	155
第 5 章 基本引用类型	103	6.3.1 历史	155
5.1 Date	103	6.3.2 ArrayBuffer	155
5.1.1 继承的方法	105	6.3.3 DataView	156
5.1.2 日期格式化方法	106	6.3.4 定型数组	159
5.1.3 日期/时间组件方法	106	6.4 Map	163
5.2 RegExp	107	6.4.1 基本 API	164
5.2.1 RegExp 实例属性	109	6.4.2 顺序与迭代	166
5.2.2 RegExp 实例方法	109	6.4.3 选择 Object 还是 Map	168
5.2.3 RegExp 构造函数属性	111	6.5 WeakMap	168
5.2.4 模式局限	113	6.5.1 基本 API	168
5.3 原始值包装类型	113	6.5.2 弱键	170
5.3.1 Boolean	114	6.5.3 不可迭代键	170
		6.5.4 使用弱映射	171

6.6	Set	173	8.2.3	构造函数模式	221
6.6.1	基本 API	173	8.2.4	原型模式	224
6.6.2	顺序与迭代	175	8.2.5	对象迭代	233
6.6.3	定义正式集合操作	176	8.3	继承	238
6.7	WeakSet	178	8.3.1	原型链	238
6.7.1	基本 API	178	8.3.2	盗用构造函数	243
6.7.2	弱值	179	8.3.3	组合继承	244
6.7.3	不可迭代值	180	8.3.4	原型式继承	245
6.7.4	使用弱集合	180	8.3.5	寄生式继承	246
6.8	迭代与扩展操作	180	8.3.6	寄生式组合继承	247
6.9	小结	182	8.4	类	249
第 7 章	迭代器与生成器	183	8.4.1	类定义	249
7.1	理解迭代	183	8.4.2	类构造函数	250
7.2	迭代器模式	184	8.4.3	实例、原型和类成员	254
7.2.1	可迭代协议	184	8.4.4	继承	258
7.2.2	迭代器协议	186	8.5	小结	265
7.2.3	自定义迭代器	188	第 9 章	代理与反射	266
7.2.4	提前终止迭代器	190	9.1	代理基础	266
7.3	生成器	192	9.1.1	创建空代理	266
7.3.1	生成器基础	192	9.1.2	定义捕获器	267
7.3.2	通过 yield 中断执行	194	9.1.3	捕获器参数和反射 API	268
7.3.3	生成器作为默认迭代器	201	9.1.4	捕获器不变式	270
7.3.4	提前终止生成器	202	9.1.5	可撤销代理	271
7.4	小结	204	9.1.6	实用反射 API	271
第 8 章	对象、类与面向对象编程	205	9.1.7	代理另一个代理	273
8.1	理解对象	205	9.1.8	代理的问题与不足	273
8.1.1	属性的类型	206	9.2	代理捕获器与反射方法	274
8.1.2	定义多个属性	208	9.2.1	get()	275
8.1.3	读取属性的特性	209	9.2.2	set()	275
8.1.4	合并对象	210	9.2.3	has()	276
8.1.5	对象标识及相等判定	213	9.2.4	defineProperty()	277
8.1.6	增强的对象语法	213	9.2.5	getOwnProperty-	
8.1.7	对象解构	216		Descriptor()	277
8.2	创建对象	220	9.2.6	deleteProperty()	278
8.2.1	概述	220	9.2.7	ownKeys()	279
8.2.2	工厂模式	220	9.2.8	getPrototypeOf()	279
			9.2.9	setPrototypeOf()	280

9.2.10	isExtensible()	280	10.16	私有变量	316
9.2.11	preventExtensions()	281	10.16.1	静态私有变量	317
9.2.12	apply()	281	10.16.2	模块模式	318
9.2.13	construct()	282	10.16.3	模块增强模式	320
9.3	代理模式	283	10.17	小结	321
9.3.1	跟踪属性访问	283	第 11 章	期约与异步函数	322
9.3.2	隐藏属性	283	11.1	异步编程	322
9.3.3	属性验证	284	11.1.1	同步与异步	322
9.3.4	函数与构造函数参数验证	284	11.1.2	以往的异步编程模式	323
9.3.5	数据绑定与可观察对象	285	11.2	期约	325
9.4	小结	286	11.2.1	Promises/A+规范	325
第 10 章	函数	287	11.2.2	期约基础	325
10.1	箭头函数	288	11.2.3	期约的实例方法	329
10.2	函数名	289	11.2.4	期约连锁与期约合成	338
10.3	理解参数	290	11.2.5	期约扩展	345
10.4	没有重载	292	11.3	异步函数	347
10.5	默认参数值	293	11.3.1	异步函数	348
10.6	参数扩展与收集	295	11.3.2	停止和恢复执行	353
10.6.1	扩展参数	295	11.3.3	异步函数策略	356
10.6.2	收集参数	296	11.4	小结	360
10.7	函数声明与函数表达式	297	第 12 章	BOM	361
10.8	函数作为值	297	12.1	window 对象	361
10.9	函数内部	299	12.1.1	Global 作用域	361
10.9.1	arguments	299	12.1.2	窗口关系	362
10.9.2	this	300	12.1.3	窗口位置与像素比	362
10.9.3	caller	301	12.1.4	窗口大小	363
10.9.4	new.target	301	12.1.5	视口位置	364
10.10	函数属性与方法	302	12.1.6	导航与打开新窗口	365
10.11	函数表达式	304	12.1.7	定时器	368
10.12	递归	306	12.1.8	系统对话框	370
10.13	尾调用优化	307	12.2	location 对象	372
10.13.1	尾调用优化的条件	307	12.2.1	查询字符串	372
10.13.2	尾调用优化的代码	309	12.2.2	操作地址	373
10.14	闭包	309	12.3	navigator 对象	375
10.14.1	this 对象	312	12.3.1	检测插件	376
10.14.2	内存泄漏	314	12.3.2	注册处理程序	378
10.15	立即调用的函数表达式	314			

12.4	screen 对象	379	14.3.2	MutationObserverInit 与观察范围	437
12.5	history 对象	379	14.3.3	异步回调与记录队列	442
12.5.1	导航	379	14.3.4	性能、内存与垃圾回收	443
12.5.2	历史状态管理	380	14.4	小结	444
12.6	小结	381	第 15 章	DOM 扩展	445
第 13 章	客户端检测	382	15.1	Selectors API	445
13.1	能力检测	382	15.1.1	querySelector()	445
13.1.1	安全能力检测	383	15.1.2	querySelectorAll()	446
13.1.2	基于能力检测进行浏览器 分析	384	15.1.3	matches()	447
13.2	用户代理检测	386	15.2	元素遍历	447
13.2.1	用户代理的历史	386	15.3	HTML5	448
13.2.2	浏览器分析	392	15.3.1	CSS 类扩展	448
13.3	软件与硬件检测	394	15.3.2	焦点管理	450
13.3.1	识别浏览器与操作系统	394	15.3.3	HTMLDocument 扩展	450
13.3.2	浏览器元数据	395	15.3.4	字符集属性	451
13.3.3	硬件	400	15.3.5	自定义数据属性	451
13.4	小结	400	15.3.6	插入标记	452
第 14 章	DOM	401	15.3.7	scrollIntoView()	456
14.1	节点层级	401	15.4	专有扩展	456
14.1.1	Node 类型	402	15.4.1	children 属性	456
14.1.2	Document 类型	407	15.4.2	contains() 方法	457
14.1.3	Element 类型	414	15.4.3	插入标记	457
14.1.4	Text 类型	420	15.4.4	滚动	459
14.1.5	Comment 类型	423	15.5	小结	459
14.1.6	CDATASection 类型	423	第 16 章	DOM2 和 DOM3	460
14.1.7	DocumentType 类型	424	16.1	DOM 的演进	460
14.1.8	DocumentFragment 类型	424	16.1.1	XML 命名空间	461
14.1.9	Attr 类型	425	16.1.2	其他变化	464
14.2	DOM 编程	426	16.2	样式	467
14.2.1	动态脚本	426	16.2.1	存取元素样式	467
14.2.2	动态样式	428	16.2.2	操作样式表	470
14.2.3	操作表格	429	16.2.3	元素尺寸	472
14.2.4	使用 NodeList	431	16.3	遍历	476
14.3	MutationObserver 接口	432	16.3.1	NodeIterator	478
14.3.1	基本用法	433	16.3.2	TreeWalker	480

16.4	范围	481	17.5	内存与性能	540
16.4.1	DOM 范围	482	17.5.1	事件委托	540
16.4.2	简单选择	482	17.5.2	删除事件处理程序	541
16.4.3	复杂选择	483	17.6	模拟事件	543
16.4.4	操作范围	484	17.6.1	DOM 事件模拟	543
16.4.5	范围插入	486	17.6.2	IE 事件模拟	547
16.4.6	范围折叠	487	17.7	小结	548
16.4.7	范围比较	488	第 18 章	动画与 Canvas 图形	549
16.4.8	复制范围	489	18.1	使用 requestAnimationFrame	549
16.4.9	清理	489	18.1.1	早期定时动画	549
16.5	小结	489	18.1.2	时间间隔的问题	550
第 17 章	事件	490	18.1.3	requestAnimationFrame	550
17.1	事件流	490	18.1.4	cancelAnimationFrame	551
17.1.1	事件冒泡	490	18.1.5	通过 requestAnimation- Frame 节流	551
17.1.2	事件捕获	491	18.2	基本的画布功能	552
17.1.3	DOM 事件流	492	18.3	2D 绘图上下文	553
17.2	事件处理程序	493	18.3.1	填充和描边	554
17.2.1	HTML 事件处理程序	493	18.3.2	绘制矩形	554
17.2.2	DOM0 事件处理程序	495	18.3.3	绘制路径	556
17.2.3	DOM2 事件处理程序	495	18.3.4	绘制文本	558
17.2.4	IE 事件处理程序	497	18.3.5	变换	560
17.2.5	跨浏览器事件处理程序	498	18.3.6	绘制图像	562
17.3	事件对象	499	18.3.7	阴影	563
17.3.1	DOM 事件对象	499	18.3.8	渐变	564
17.3.2	IE 事件对象	502	18.3.9	图案	566
17.3.3	跨浏览器事件对象	503	18.3.10	图像数据	566
17.4	事件类型	505	18.3.11	合成	567
17.4.1	用户界面事件	506	18.4	WebGL	569
17.4.2	焦点事件	510	18.4.1	WebGL 上下文	569
17.4.3	鼠标和滚轮事件	510	18.4.2	WebGL 基础	569
17.4.4	键盘与输入事件	518	18.4.3	WebGL1 与 WebGL2	579
17.4.5	合成事件	522	18.5	小结	579
17.4.6	变化事件	523	第 19 章	表单脚本	581
17.4.7	HTML5 事件	523	19.1	表单基础	581
17.4.8	设备事件	528	19.1.1	提交表单	582
17.4.9	触摸及手势事件	531	19.1.2	重置表单	583
17.4.10	事件参考	534			

19.1.3 表单字段	583	20.5.4 检测编解码器	630
19.2 文本框编程	587	20.5.5 音频类型	631
19.2.1 选择文本	588	20.6 原生拖放	631
19.2.2 输入过滤	590	20.6.1 拖放事件	631
19.2.3 自动切换	593	20.6.2 自定义放置目标	632
19.2.4 HTML5 约束验证 API	594	20.6.3 dataTransfer 对象	632
19.3 选择框编程	597	20.6.4 dropEffect 与 effectAllowed	633
19.3.1 选项处理	598	20.6.5 可拖动能力	634
19.3.2 添加选项	599	20.6.6 其他成员	634
19.3.3 移除选项	600	20.7 Notifications API	635
19.3.4 移动和重排选项	601	20.7.1 通知权限	635
19.4 表单序列化	601	20.7.2 显示和隐藏通知	635
19.5 富文本编辑	603	20.7.3 通知生命周期回调	636
19.5.1 使用 contentEditable	603	20.8 Page Visibility API	636
19.5.2 与富文本交互	604	20.9 Streams API	637
19.5.3 富文件选择	606	20.9.1 理解流	637
19.5.4 通过表单提交富文本	607	20.9.2 可读流	638
19.6 小结	608	20.9.3 可写流	640
第 20 章 JavaScript API	609	20.9.4 转换流	641
20.1 Atomics 与 SharedArrayBuffer	609	20.9.5 通过管道连接流	642
20.1.1 SharedArrayBuffer	610	20.10 计时 API	644
20.1.2 原子操作基础	611	20.10.1 High Resolution Time API	644
20.2 跨上下文消息	616	20.10.2 Performance Timeline API	645
20.3 Encoding API	617	20.11 Web 组件	648
20.3.1 文本编码	617	20.11.1 HTML 模板	648
20.3.2 文本解码	619	20.11.2 影子 DOM	651
20.4 File API 与 Blob API	622	20.11.3 自定义元素	657
20.4.1 File 类型	622	20.12 Web Cryptography API	663
20.4.2 FileReader 类型	622	20.12.1 生成随机数	663
20.4.3 FileReaderSync 类型	624	20.12.2 使用 SubtleCrypto 对象	664
20.4.4 Blob 与部分读取	624	20.13 小结	674
20.4.5 对象 URL 与 Blob	625	第 21 章 错误处理与调试	675
20.4.6 读取拖放文件	626	21.1 浏览器错误报告	675
20.5 媒体元素	627		
20.5.1 属性	627		
20.5.2 事件	628		
20.5.3 自定义媒体播放器	629		

21.1.1 桌面控制台	675	22.3.2 使用参数	701
21.1.2 移动控制台	676	22.3.3 重置处理器	702
21.2 错误处理	676	22.4 小结	702
21.2.1 try/catch 语句	676	第 23 章 JSON	703
21.2.2 抛出错误	679	23.1 语法	703
21.2.3 error 事件	681	23.1.1 简单值	703
21.2.4 错误处理策略	682	23.1.2 对象	704
21.2.5 识别错误	682	23.1.3 数组	704
21.2.6 区分重大与非重大错误	686	23.2 解析与序列化	706
21.2.7 把错误记录到服务器中	687	23.2.1 JSON 对象	706
21.3 调试技术	688	23.2.2 序列化选项	707
21.3.1 把消息记录到控制台	688	23.2.3 解析选项	710
21.3.2 理解控制台运行时	689	23.3 小结	710
21.3.3 使用 JavaScript 调试器	689	第 24 章 网络请求与远程资源	711
21.3.4 在页面中打印消息	690	24.1 XMLHttpRequest 对象	711
21.3.5 补充控制台方法	690	24.1.1 使用 XHR	712
21.3.6 抛出错误	690	24.1.2 HTTP 头部	713
21.4 旧版 IE 的常见错误	691	24.1.3 GET 请求	715
21.4.1 无效字符	691	24.1.4 POST 请求	715
21.4.2 未找到成员	692	24.1.5 XMLHttpRequest Level 2	716
21.4.3 未知运行时错误	692	24.2 进度事件	718
21.4.4 语法错误	692	24.2.1 load 事件	718
21.4.5 系统找不到指定资源	693	24.2.2 progress 事件	719
21.5 小结	693	24.3 跨源资源共享	719
第 22 章 处理 XML	694	24.3.1 预检请求	720
22.1 浏览器对 XML DOM 的支持	694	24.3.2 凭据请求	721
22.1.1 DOM Level 2 Core	694	24.4 替代性跨源技术	721
22.1.2 DOMParser 类型	695	24.4.1 图片探测	721
22.1.3 XMLSerializer 类型	696	24.4.2 JSONP	722
22.2 浏览器对 XPath 的支持	696	24.5 Fetch API	722
22.2.1 DOM Level 3 XPath	696	24.5.1 基本用法	723
22.2.2 单个节点结果	698	24.5.2 常见 Fetch 请求模式	728
22.2.3 简单类型结果	698	24.5.3 Headers 对象	730
22.2.4 默认类型结果	699	24.5.4 Request 对象	732
22.2.5 命名空间支持	699	24.5.5 Response 对象	735
22.3 浏览器对 XSLT 的支持	700	24.5.6 Request、Response 及 Body 混入	739
22.3.1 XSLTProcessor 类型	700		

24.6	Beacon API	747	26.1.4	入口	773
24.7	Web Socket	747	26.1.5	异步依赖	774
24.7.1	API	748	26.1.6	动态依赖	774
24.7.2	发送和接收数据	748	26.1.7	静态分析	774
24.7.3	其他事件	748	26.1.8	循环依赖	775
24.8	安全	749	26.2	凑合的模块系统	776
24.9	小结	750	26.3	使用 ES6 之前的模块加载器	779
第 25 章	客户端存储	751	26.3.1	CommonJS	779
25.1	cookie	751	26.3.2	异步模块定义	781
25.1.1	限制	751	26.3.3	通用模块定义	782
25.1.2	cookie 的构成	752	26.3.4	模块加载器终将没落	782
25.1.3	JavaScript 中的 cookie	753	26.4	使用 ES6 模块	783
25.1.4	子 cookie	755	26.4.1	模块标签及定义	783
25.1.5	使用 cookie 的注意事项	759	26.4.2	模块加载	784
25.2	Web Storage	759	26.4.3	模块行为	784
25.2.1	Storage 类型	759	26.4.4	模块导出	785
25.2.2	sessionStorage 对象	760	26.4.5	模块导入	787
25.2.3	localStorage 对象	761	26.4.6	模块转移导出	789
25.2.4	存储事件	762	26.4.7	工作者模块	789
25.2.5	限制	762	26.4.8	向后兼容	790
25.3	IndexedDB	762	26.5	小结	790
25.3.1	数据库	763	第 27 章	工作者线程	791
25.3.2	对象存储	763	27.1	工作者线程简介	791
25.3.3	事务	764	27.1.1	工作者线程与线程	791
25.3.4	插入对象	765	27.1.2	工作者线程的类型	792
25.3.5	通过游标查询	765	27.1.3	WorkerGlobalScope	793
25.3.6	键范围	767	27.2	专用工作者线程	793
25.3.7	设置游标方向	768	27.2.1	专用工作者线程的基本 概念	794
25.3.8	索引	769	27.2.2	专用工作者线程与隐式 MessagePorts	796
25.3.9	并发问题	770	27.2.3	专用工作者线程的生命 周期	796
25.3.10	限制	771	27.2.4	配置 Worker 选项	798
25.4	小结	771	27.2.5	在 JavaScript 行内创建 工作者线程	798
第 26 章	模块	772	27.2.6	在工作者线程中动态执行 脚本	799
26.1	理解模块模式	772			
26.1.1	模块标识符	772			
26.1.2	模块依赖	773			
26.1.3	模块加载	773			

27.2.7	委托任务到子工作者线程	800
27.2.8	处理工作者线程错误	801
27.2.9	与专用工作者线程通信	801
27.2.10	工作者线程数据传输	805
27.2.11	线程池	810
27.3	共享工作者线程	813
27.3.1	共享工作者线程简介	813
27.3.2	理解共享工作者线程的 生命周期	815
27.3.3	连接到共享工作者线程	816
27.4	服务工作者线程	817
27.4.1	服务工作者线程基础	818
27.4.2	服务工作者线程缓存	824
27.4.3	服务工作者线程客户端	829
27.4.4	服务工作者线程与一致性	829
27.4.5	理解服务工作者线程的 生命周期	830
27.4.6	控制反转与服务工作者 线程持久化	834
27.4.7	通过 updateViaCache 管理服务文件缓存	835
27.4.8	强制性服务工作者线程 操作	835
27.4.9	服务工作者线程消息	836
27.4.10	拦截 fetch 事件	837

27.4.11	推送通知	839
27.5	小结	841

第 28 章 最佳实践

28.1	可维护性	842
28.1.1	什么是可维护的代码	842
28.1.2	编码规范	843
28.1.3	松散耦合	845
28.1.4	编码惯例	848
28.2	性能	851
28.2.1	作用域意识	851
28.2.2	选择正确的方法	852
28.2.3	语句最少化	857
28.2.4	优化 DOM 交互	858
28.3	部署	861
28.3.1	构建流程	861
28.3.2	验证	862
28.3.3	压缩	863
28.4	小结	864

附录 A ES2018 和 ES2019 (图灵社区下载)

附录 B 严格模式 (图灵社区下载)

附录 C JavaScript 库和框架 (图灵社区下载)

附录 D JavaScript 工具 (图灵社区下载)

第 1 章

什么是 JavaScript

本章内容

- JavaScript 历史回顾
- JavaScript 是什么
- JavaScript 与 ECMAScript 的关系
- JavaScript 的不同版本

1995 年，JavaScript 问世。当时，它的主要用途是代替 Perl 等服务器端语言处理输入验证。在此之前，要验证某个必填字段是否已填写，或者某个输入的值是否有效，需要与服务器的往返通信。网景公司希望通过在其 Navigator 浏览器中加入 JavaScript 来改变这个局面。在那个普遍通过电话拨号上网的年代，由客户端处理某些基本的验证是让人兴奋的新功能。缓慢的网速让页面每次刷新都考验着人们的耐心。

从那时起，JavaScript 逐渐成为市面上所有主流浏览器的标配。如今，JavaScript 的应用也不再局限于数据验证，而是渗透到浏览器窗口及其内容的方方面面。JavaScript 已被公认为主流的编程语言，能够实现复杂的计算与交互，包括闭包、匿名（lambda）函数，甚至元编程等特性。不仅是桌面浏览器，手机浏览器和屏幕阅读器也支持 JavaScript，其重要性可见一斑。就连拥有自家客户端脚本语言 VBScript 的微软公司，也在其 Internet Explorer（以下简称 IE）浏览器最初的版本中包含了自己的 JavaScript 实现。

从简单的输入验证脚本到强大的编程语言，JavaScript 的崛起没有任何人预测到。它很简单，学会用只要几分钟；它又很复杂，掌握它要很多年。要真正学好用好 JavaScript，理解其本质、历史及局限性是非常重要的。

1.1 简短的历史回顾

随着 Web 日益流行，对客户端脚本语言的需求也越来越强烈。当时，大多数用户使用 28.8kbit/s 的调制解调器上网，但网页变得越来越大、越来越复杂。为验证简单的表单而需要大量与服务器的往返通信成为用户的痛点。想象一下，你填写完表单，单击“提交”按钮，等 30 秒处理，然后看到一条消息，告诉你有一个必填字段没填。网景在当时是引领技术革新的公司，它将开发一个客户端脚本语言来处理这种简单的数据验证提上了日程。

1995 年，网景公司一位名叫 Brendan Eich 的工程师，开始为即将发布的 Netscape Navigator 2 开发一个叫 Mocha（后来改名为 LiveScript）的脚本语言。当时的计划是在客户端和服务端都使用它，它在服务端叫 LiveWire。

为了赶上发布时间，网景与 Sun 公司结为开发联盟，共同完成 LiveScript 的开发。就在 Netscape Navigator 2 正式发布前，网景把 LiveScript 改名为 JavaScript，以便搭上媒体当时热烈炒作 Java 的顺风车。

由于 JavaScript 1.0 很成功,网景又在 Netscape Navigator 3 中发布了 1.1 版本。尚未成熟的 Web 的受欢迎程度达到了历史新高,而网景则稳居市场领导者的位置。这时候,微软决定向 IE 投入更多资源。就在 Netscape Navigator 3 发布后不久,微软发布了 IE3,其中包含自己名为 JScript (叫这个名字是为了避免与网景发生许可纠纷)的 JavaScript 实现。1996 年 8 月,微软重磅进入 Web 浏览器领域,这是网景永远的痛,但它代表 JavaScript 作为一门语言向前迈进了一大步。

微软的 JavaScript 实现意味着出现了两个版本的 JavaScript: Netscape Navigator 中的 JavaScript,以及 IE 中的 JScript。与 C 语言以及很多其他编程语言不同,JavaScript 还没有规范其语法或特性的标准,两个版本并存让这个问题更加突出了。随着业界担忧日甚,JavaScript 终于踏上了标准化的征程。

1997 年,JavaScript 1.1 作为提案被提交给欧洲计算机制造商协会(Ecma)。第 39 技术委员会(TC39)承担了“标准化一门通用、跨平台、厂商中立的脚本语言的语法和语义”的任务(参见 TC39-ECMAScript)。TC39 委员会由来自网景、Sun、微软、Borland、Nombas 和其他对这门脚本语言有兴趣的公司的工程师组成。他们花了数月时间打造出 ECMA-262,也就是 ECMAScript (发音为“ek-ma-script”)这个新的脚本语言标准。

1998 年,国际标准化组织(ISO)和国际电工委员会(IEC)也将 ECMAScript 采纳为标准(ISO/IEC-16262)。自此以后,各家浏览器均以 ECMAScript 作为自己 JavaScript 实现的依据,虽然具体实现各有不同。

1.2 JavaScript 实现

虽然 JavaScript 和 ECMAScript 基本上是同义词,但 JavaScript 远远不限于 ECMA-262 所定义的那样。没错,完整的 JavaScript 实现包含以下几个部分(见图 1-1):

- ❑ 核心(ECMAScript)
- ❑ 文档对象模型(DOM)
- ❑ 浏览器对象模型(BOM)

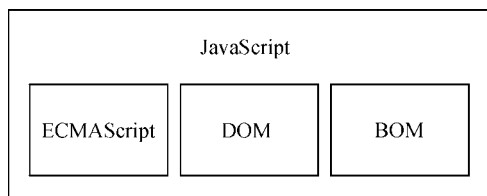


图 1-1

1.2.1 ECMAScript

ECMAScript,即 ECMA-262 定义的语言,并不局限于 Web 浏览器。事实上,这门语言没有输入和输出之类的方法。ECMA-262 将这门语言作为一个基准来定义,以便在它之上再构建更稳健的脚本语言。Web 浏览器只是 ECMAScript 实现可能存在的一种宿主环境(host environment)。宿主环境提供 ECMAScript 的基准实现和与环境自身交互必需的扩展。扩展(比如 DOM)使用 ECMAScript 核心类型和语法,提供特定于环境的额外功能。其他宿主环境还有服务器端 JavaScript 平台 Node.js 和即将被淘汰的 Adobe Flash。

如果不涉及浏览器的话, ECMA-262 到底定义了什么? 在基本的层面, 它描述这门语言的如下部分:

- ❑ 语法
- ❑ 类型
- ❑ 语句
- ❑ 关键字
- ❑ 保留字
- ❑ 操作符
- ❑ 全局对象

ECMAScript 只是对实现这个规范描述的所有方面的一门语言的称呼。JavaScript 实现了 ECMAScript, 而 Adobe ActionScript 同样也实现了 ECMAScript。

1. ECMAScript 版本

ECMAScript 不同的版本以 “edition” 表示 (也就是描述特定实现的 ECMA-262 的版本)。ECMA-262 最近的版本是第 10 版, 发布于 2019 年 6 月。ECMA-262 的第 1 版本质上跟网景的 JavaScript 1.1 相同, 只不过删除了所有浏览器特定的代码, 外加少量细微的修改。ECMA-262 要求支持 Unicode 标准 (以支持多语言), 而且对象要与平台无关 (Netscape JavaScript 1.1 的对象不是这样, 比如它的 Date 对象就依赖平台)。这也是 JavaScript 1.1 和 JavaScript 1.2 不符合 ECMA-262 第 1 版要求的原因。

ECMA-262 第 2 版只是做了一些编校工作, 主要是为了更新之后严格符合 ISO/IEC-16262 的要求, 并没有增减或改变任何特性。ECMAScript 实现通常不使用第 2 版来衡量符合性 (conformance)。

ECMA-262 第 3 版第一次真正对这个标准进行更新, 更新了字符串处理、错误定义和数值输出。此外还增加了对正则表达式、新的控制语句、try/catch 异常处理的支持, 以及为了更好地让标准国际化所做的少量修改。对很多人来说, 这标志着 ECMAScript 作为一门真正的编程语言的时代终于到来了。

ECMA-262 第 4 版是对这门语言的一次彻底修订。作为对 JavaScript 在 Web 上日益成功的回应, 开发者开始修订 ECMAScript 以满足全球 Web 开发日益增长的需求。为此, Ecma T39 再次被召集起来, 以决定这门语言的未来。结果, 他们制定的规范几乎在第 3 版基础上完全定义了一门新语言。第 4 版包括强类型变量、新语句和数据结构、真正的类和经典的继承, 以及操作数据的新手段。

与此同时, TC39 委员会的一个子委员会也提出了另外一份提案, 叫作 “ECMAScript 3.1”, 只对这门语言进行了较少的改进。这个子委员会的人认为第 4 版对这门语言来说跳跃太大了。因此, 他们提出了一个改动较小的提案, 只要在现有 JavaScript 引擎基础上做一些增改就可以实现。最终, ES3.1 子委员会赢得了 TC39 委员会的支持, ECMA-262 第 4 版在正式发布之前被放弃。

ECMAScript 3.1 变成了 ECMA-262 的第 5 版, 于 2009 年 12 月 3 日正式发布。第 5 版致力于厘清第 3 版存在的歧义, 也增加了新功能。新功能包括原生的解析和序列化 JSON 数据的 JSON 对象、方便继承和高级属性定义的方法, 以及新的增强 ECMAScript 引擎解释和执行代码能力的严格模式。第 5 版在 2011 年 6 月发布了一个维护性修订版, 这个修订版只更正了规范中的错误, 并未增加任何新的语言或库特性。

ECMA-262 第 6 版, 俗称 ES6、ES2015 或 ES Harmony (和谐版), 于 2015 年 6 月发布。这一版包含了大概这个规范有史以来最重要的一批增强特性。ES6 正式支持了类、模块、迭代器、生成器、箭头函数、期约、反射、代理和众多新的数据类型。

ECMA-262 第 7 版, 也称为 ES7 或 ES2016, 于 2016 年 6 月发布。这次修订只包含少量语法层面的增强, 如 `Array.prototype.includes` 和指数操作符。

ECMA-262 第 8 版, 也称为 ES8、ES2017, 完成于 2017 年 6 月。这一版主要增加了异步函数 (async/await)、SharedArrayBuffer 及 Atomics API, 以及 Object.values()/Object.entries()/Object.getOwnPropertyDescriptors() 和字符串填充方法, 另外明确支持对象字面量最后的逗号。

ECMA-262 第 9 版, 也称为 ES9、ES2018, 发布于 2018 年 6 月。这次修订包括异步迭代、剩余和扩展属性、一组新的正则表达式特性、Promise finally(), 以及模板字面量修订。

ECMA-262 第 10 版, 也称为 ES10、ES2019, 发布于 2019 年 6 月。这次修订增加了 Array.prototype.flat()/flatMap()、String.prototype.trimStart()/trimEnd()、Object.fromEntries() 方法, 以及 Symbol.prototype.description 属性, 明确定义了 Function.prototype.toString() 的返回值并固定了 Array.prototype.sort() 的顺序。另外, 这次修订解决了与 JSON 字符串兼容的问题, 并定义了 catch 子句的可选绑定。

2. ECMAScript 符合性是什么意思

ECMA-262 阐述了什么是 ECMAScript 符合性。要成为 ECMAScript 实现, 必须满足下列条件:

- ❑ 支持 ECMA-262 中描述的所有“类型、值、对象、属性、函数, 以及程序语法与语义”;
- ❑ 支持 Unicode 字符标准。

此外, 符合性实现还可以满足下列要求。

- ❑ 增加 ECMA-262 中未提及的“额外的类型、值、对象、属性和函数”。ECMA-262 所说的这些额外内容主要指规范中未给出的新对象或对象的新属性。
- ❑ 支持 ECMA-262 中没有定义的“程序和正则表达式语法”(意思是允许修改和扩展内置的正则表达式特性)。

以上条件为实现开发者基于 ECMAScript 开发语言提供了极大的权限和灵活度, 也是其广受欢迎的原因之一。

3. 浏览器对 ECMAScript 的支持

1996 年, Netscape Navigator 3 发布时包含了 JavaScript 1.1。JavaScript 1.1 规范随后被提交给 Ecma, 作为对新的 ECMA-262 标准的建议。随着 JavaScript 迅速走红, 网景非常愿意开发 1.2 版。可是有个问题: Ecma 尚未接受网景的建议。

Netscape Navigator 3 发布后不久, 微软推出了 IE3。IE 的这个版本包含了 JScript 1.0, 本意是提供与 JavaScript 1.1 相同的功能。不过, 由于缺少很多文档, 而且还有不少重复性功能, JScript 1.0 远远没有 JavaScript 1.1 那么强大。

JScript 的再次更新出现在 IE4 中的 JScript 3.0 (2.0 版是在 Microsoft Internet Information Server 3.0 中发布的, 但从未包含在浏览器中)。微软发新闻稿称 JScript 3.0 是世界上第一门真正兼容 Ecma 标准的脚本语言。当时 ECMA-262 还没制定完成, 因此 JScript 3.0 遭受了与 JavaScript 1.2 同样的命运, 它同样没有遵守最终的 ECMAScript 标准。

网景又在 Netscape Navigator 4.06 中将其 JavaScript 版本升级到 1.3, 因此做到了与 ECMA-262 第 1 版完全兼容。JavaScript 1.3 增加了对 Unicode 标准的支持, 并做到了所有对象都与平台无关, 同时保留了 JavaScript 1.2 所有的特性。

后来, 当网景以 Mozilla 项目的名义向公众发布其源代码时, 人们都期待 Netscape Navigator 5 中会包含 JavaScript 1.4。可是, 一个完全重新设计网景代码的激进决定导致了人们的希望落空。JavaScript 1.4 只在 Netscape Enterprise Server 中作为服务器端语言发布了, 从来就没有进入浏览器。

到了 2008 年, 五大浏览器 (IE、Firefox、Safari、Chrome 和 Opera) 全部兼容 ECMA-262 第 3 版。

IE8 率先实现 ECMA-262 第 5 版，并在 IE9 中完整支持。Firefox 4 很快也做到了。下表列出了主要的浏览器版本对 ECMAScript 的支持情况。

浏 览 器	ECMAScript 符合性
Netscape Navigator 2	—
Netscape Navigator 3	—
Netscape Navigator 4~4.05	—
Netscape Navigator 4.06~4.79	第 1 版
Netscape 6+ (Mozilla 0.6.0+)	第 3 版
IE3	—
IE4	—
IE5	第 1 版
IE5.5~8	第 3 版
IE9	第 5 版 (部分)
IE10~11	第 5 版
Edge 12+	第 6 版
Opera 6~7.1	第 2 版
Opera 7.2+	第 3 版
Opera 15~28	第 5 版
Opera 29~35	第 6 版 (部分)
Opera 36+	第 6 版
Safari 1~2.0.x	第 3 版 (部分)
Safari 3.1~5.1	第 5 版 (部分)
Safari 6~8	第 5 版
Safari 9+	第 6 版
iOS Safari 3.2~5.1	第 5 版 (部分)
iOS Safari 6~8.4	第 5 版
iOS Safari 9.2+	第 6 版
Chrome 1~3	第 3 版
Chrome 4~22	第 5 版 (部分)
Chrome 23+	第 5 版
Chrome 42~48	第 6 版 (部分)
Chrome 49+	第 6 版
Firefox 1~2	第 3 版
Firefox 3.0.x~20	第 5 版 (部分)
Firefox 21~44	第 5 版
Firefox 45+	第 6 版